

6.2 Inline Documentation

Inline documentation includes all comments and documentation that are embedded directly in the source code of an application. This type of documentation helps to make the code easier to understand as you read it.

You should use inline comments if you want to explain individual sections of code. They are mainly used for complex code sections that are difficult to understand. A typical example is a code passage that isn't intuitive. If you've implemented a part of the business logic in such a way that it works but doesn't correspond to the code standard of your application, you should comment on this. If you were to adapt the code accordingly, the logic would no longer work as desired. To prevent you or another person from making this change at a later date, you can leave a comment.

Like almost everything in software development, inline documentation has both advantages and disadvantages. Depending on the context in which you use the documentation, these aspects come into play to a greater or lesser extent.

One advantage of inline documentation is that you can use it immediately and thus explain complex code directly. The main disadvantages are the reduced readability of the code and potential over-commenting.

As the disadvantages of inline documentation clearly outweigh the advantages, you should use it sparingly and only in areas that truly require explanation. You should formulate the remaining source code of your application in such a self-explanatory way that no inline comments are required.

6.2.1 Sample Inline Documentation

As a concrete example of the useful insertion of inline comments, we use a function that calculates the nth Fibonacci number. Without comments, the code of the function reads as follows:

```
const fibonacci = (n: number): number => {
  if (n < 0) {
    throw new Error("n must be a non-negative
number");
  }
  if (n === 0) return 0;
  if (n === 1) return 1;
  let a = 0;
  let b = 1;
  for (let i = 2; i <= n; i++) {
    const temp = a + b;
    a = b;
    b = temp;
  }
  return b;
};
```

The only indication of the function's purpose is its name. Both the code and the abbreviated variable names don't give away at first glance what the function is actually supposed to do. So in the next step, we ask ChatGPT to insert explanatory inline comments:

Prompt: Please add inline comments to the following function to explain what is happening.

[code of the function]

Sure, here is the function with inline comments explaining each step:

```
const fibonacci = (n: number): number => {
  // Check if the input is a negative number and throw
  an
  // error if it is
  if (n < 0) {
```

```

        throw new Error("n must be a non-negative
number");
    }

    // If n is 0, the Fibonacci number is 0
    if (n === 0) return 0;

    // If n is 1, the Fibonacci number is 1
    if (n === 1) return 1;

    // Initialize the first two Fibonacci numbers
    let a = 0;
    let b = 1;

    // Loop from 2 to n to calculate the nth Fibonacci
number
    for (let i = 2; i <= n; i++) {
        // Calculate the next Fibonacci number
        const temp = a + b;
        // Update the previous two Fibonacci numbers
        a = b;
        b = temp;
    }

    // Return the nth Fibonacci number
    return b;
};

```

These explanatory comments, which expound on the purpose of almost every line, make the algorithm easy to understand. Lines that belong together and are explained with a comment have also been combined here by the AI tool and separated from the rest by a blank line. This type of documentation helps you create algorithms so that they are explained directly in the code for other people or yourself in the future. However, you can also use this type of AI support if you need to maintain code that you've taken over from other people. Before you laboriously work out the meaning of the code yourself, you can have it explained to you in this way and include this explanation as part of the code in the version control.

For a meaningful inline comment, the AI tool needs the source code you want to comment on and an instruction on what exactly you want to

achieve. In this case, the task was clear, as the entire function was to be commented on step-by-step. However, you can also have only individual comments inserted. Here is an example of the calculation of a discounted price.

```
function calculateDiscount(price, discount) {  
    const validDiscount = discount > 100 ? 100 :  
    discount;  
  
    const finalPrice = price - (price * (validDiscount /  
    100));  
  
    return finalPrice;  
}
```

You can ask ChatGPT to explain to you what the line containing `validDiscount` is supposed to do and also insert a comment.

Prompt: What does the line with `validDiscount` do? Please add an inline comment.

[code of the function]

The line with `validDiscount` is checking if the discount value is greater than 100. If it is, it sets `validDiscount` to 100, effectively capping the discount at 100%. Otherwise, it keeps the discount as is. This ensures that the discount doesn't exceed 100%, which wouldn't make sense in a typical pricing scenario.

```
function calculateDiscount(price, discount) {  
    // Ensure the discount does not exceed 100%  
    // by capping it at 100  
    const validDiscount = discount > 100 ? 100 :  
    discount;  
    ...  
}
```

[more details]

You'll have to deal with such cases much more frequently than with commenting on each individual line. In this case, however, you also need to give your AI tool more information about what exactly you expect.

You should use inline documentation sparingly, as it doesn't always improve the readability of the source code. Function and class documentation is used much more frequently.